

Sound Demand-Driven Alias Analysis for LLVM

Future LLVM GSoC application | Large project (~350 hours, 12 weeks) | Yogesh Chavan

1. Synopsis

LLVM optimizations often ask whether two memory locations can refer to the same storage. When existing analyses return MayAlias, an optimization may have to act conservatively. I propose an opt-in, function-local alias analysis that uses a demand-driven pointer-constraint graph to answer a useful subset of these queries. It will use safe graph summaries to reject impossible candidates cheaply and bounded graph search for harder cases. A budget limit or unsupported construct will return MayAlias, never an unjustified NoAlias. The result will be a tested LLVM prototype, a reproducible evaluation, and a design report explaining when the analysis helps and when it does not.

The earlier 2015 draft proposed replacing grammar-constrained search with set lookups. This revision treats that as a research hypothesis, not a correctness claim: set membership alone cannot represent every may-alias relation. Any summary used to produce NoAlias must be justified by a soundness argument and checked against a reference implementation on the supported subset.

2. Motivation and benefit to LLVM

The value of this work is a measured answer to a practical question: can a more selective, demand-driven analysis improve useful NoAlias answers without unacceptable compile-time or memory cost? LLVM already has BasicAA and an AA aggregation interface; this project is intended to complement that system through explicit registration and comparison, not to replace LLVM's default pipeline. The community would receive reusable correctness cases, benchmark scripts, and a documented prototype even if the measured tradeoff argues against enabling the analysis by default. [1][2]

3. Related work and project boundary

Prior CFL-reachability approaches motivate graph-based, demand-driven queries, while LLVM's current AAResults/AAManager provides the integration point. Existing BasicAA and the configured default AA pipeline are the baselines. LLVM also provides aa-eval for precision measurement and lit/FileCheck for regression tests. The project will first check the current source tree and mentor feedback for overlapping implementations and existing query caches; it will not claim novelty or a speedup from caching without that comparison. [1][2][3]

Required scope: intraprocedural, flow-insensitive queries over ordinary pointer values and memory locations; direct assignments/casts, select/phi, loads/stores, and constant-offset GEP where semantics can be modeled soundly. Conservative boundary: unknown external calls, pointer escape, hard indirect calls, unsupported address spaces or operations, and uncertain access sizes return MayAlias or delegate to the existing AA chain. Interprocedural context sensitivity and field-sensitive heap modeling are optional follow-ons.

4. Technical approach

4.1 Baseline and query corpus

Instrument the current AA pipeline in an opt-in evaluation build to record query counts, repeated queries, answer distributions, query time, and memory use on a fixed corpus. Include small LLVM IR tests and several open-source C/C++ programs selected with mentor input. Record exact LLVM revision, compiler flags, machine, and corpus revision. This establishes whether demand-driven work addresses an observed gap.

4.2 Analysis model and soundness gate

Build a per-function graph of supported pointer operations. Specify graph nodes, edge labels, transfer rules, memory-location sizes, and the relation between a graph answer and LLVM's NoAlias/MayAlias result in a short design note. A simple reference solver will serve as the correctness oracle for the supported subset. The optimized solver will use coarse summaries only to prune candidates when the necessary invariant is proven; otherwise it will search. NoAlias is permitted only after an exhaustive supported-case proof of non-aliasing. Timeout, budget exhaustion, missing modeling, or uncertainty returns MayAlias. The analysis will not infer MustAlias or PartialAlias in the required version. [1]

4.3 LLVM integration and iteration

Implement the analysis through the current pass-manager AA result interface as an opt-in component, with correct invalidation/lifetime behavior and a narrow diagnostic mode for evaluation. Keep implementation patches small and reviewable. After a correct reference version exists, measure one optimization at a time: summary-based filtering, memoization where it beats existing facilities, and search ordering. Remove any optimization that loses soundness or has no measured benefit. [1][2]

5. Deliverables and success criteria

- Required D1: mentor-reviewed design note with explicit supported semantics, soundness invariant, and fallback rules.
- Required D2: opt-in LLVM analysis prototype plus regression tests covering supported constructs and conservative fallbacks.
- Required D3: reproducible benchmark harness, raw results, and comparison with BasicAA/default AA for precision, query cost, end-to-end compile time, and peak memory.
- Required D4: user/developer documentation, final report, and reviewable patch series or repository branch. Upstream merge is a goal, not a promise dependent on reviewers' schedules.
- Optional: field sensitivity for constant struct offsets, limited direct-call summaries, and an end-to-end optimization study if correctness and performance gates are met.

Correctness gate: zero incorrect NoAlias answers in targeted regression tests and generated small-program comparisons; all unsupported cases must fall back conservatively. Evaluation targets: report the fraction of baseline MayAlias queries converted to sound NoAlias, median/p95 query latency, total compilation time, and peak memory. A useful outcome would be at least 5% additional NoAlias answers on the chosen corpus with at most 5% median compile-time overhead; these are hypotheses to test, not conditions for reporting an honest negative result.

6. Timeline (~350 hours)

Period	Work and reviewable output
Pre-application	Discuss idea on LLVM Discourse; confirm mentor interest, current overlap, scope, and project size. Build LLVM and submit a small self-authored contribution before applying.
Community bonding	Freeze corpus, baseline revision, weekly check-in schedule, and design questions with mentor.
Week 1	Baseline measurement harness and query corpus; initial result tables.
Week 2	Graph semantics and soundness note; mentor go/no-go on proposed pruning rules.
Week 3	Reference solver for a minimal pointer-operation subset; unit and IR tests.
Week 4	Expand supported graph operations; conservative unknown-call and escape behavior.
Week 5	Opt-in AAResults/AAManager integration; invalidation and diagnostic tests.
Week 6 / midterm	End-to-end reference prototype, correctness evidence, baseline comparison, midterm demo.
Week 7	Implement one proven summary/filter optimization; differential tests.
Week 8	Optimize search and memory layout only where profiles justify it; repeat benchmarks.
Week 9	Large-corpus evaluation; investigate every changed NoAlias answer and regression.
Week 10	Split into reviewable patches; documentation and performance analysis.
Week 11	Address mentor/reviewer feedback; run full test matrix and final benchmarks.
Week 12 / final	Final report, reproducibility package, code handoff, and LLVM project presentation/blog requirements if applicable.

7. Testing and evaluation methodology

- Soundness: lit/FileCheck regression tests for distinct objects, same object, GEP and access-size overlap, phi/select, pointer loads/stores, null/undef/poison behavior under current LLVM semantics, escapes, external and indirect calls, and unsupported IR. Compare optimized and reference solvers on generated small functions; any disagreement is a blocker for NoAlias.
- Precision: run aa-eval and a fixed query harness on the same IR with BasicAA/default AA and the opt-in analysis. Report answer transitions (especially MayAlias to NoAlias), not just a single percentage; manually inspect representative gains and losses.
- Performance: use release builds and a pinned machine. Run each corpus case at least five times after warm-up; report median and p95 query latency, graph-build time, total compile time, and peak resident memory. Separate setup cost from per-query cost and include corpus size and variance.
- Integration: run focused tests for every patch and the relevant LLVM test suites before final submission; verify no default-pipeline behavior change while the analysis remains opt-in. Use a second build configuration when practical to detect assertions or sanitizer failures. [3]

8. Risks and fallback plan

- Set/filter summary is unsound or unhelpful: retain the correct reference solver and publish the negative result; do not emit NoAlias from the summary.
- Runtime cost is high: cap work per query, keep MayAlias fallback, profile hot cases, and reduce optional scope before expanding modeling.
- LLVM API/review complexity: land the benchmark and test infrastructure first, then submit small implementation patches. A functional opt-in prototype plus reproducible results remains a complete project outcome even if merge timing extends past GSoC.

9. Applicant, communication, and submission details

Applicant: Yogesh Chavan (from the original document; verify current preferred name, email, time zone, and GitHub URL).

Relevant experience: [Applicant to write a truthful 3-5 sentence summary of current C++/compiler/LLVM experience and link personally authored work.] Availability: [Confirm approximately 30 hours/week for a large project and disclose exams, travel, employment, or other conflicts.] No prior LLVM contribution or prototype has been claimed in this draft.

Communication: propose one weekly mentor check-in, a brief written progress update with links to tests and patches, and early escalation when a correctness gate fails or a milestone slips. Before submission, seek LLVM community feedback on the exact project scope; LLVM explicitly asks for community feedback on proposals. [4]

AI-assistance disclosure: This text is an AI-assisted draft. The applicant must personally verify every technical claim, edit it into their own understood plan, and disclose substantial tool-generated content as LLVM policy requires for proposals. [5]

References

[1] LLVM, Alias Analysis Infrastructure: <https://llvm.org/docs/AliasAnalysis.html>

[2] LLVM, AAResults / AAManager and New Pass Manager: https://llvm.org/doxygen/classllvm_1_1AAResults.html ; <https://llvm.org/docs/NewPassManager.html>

[3] LLVM, Testing Infrastructure Guide and Analysis Passes: <https://llvm.org/docs/TestingGuide.html> ; <https://llvm.org/docs/Passes.html>

[4] LLVM, Open Projects / GSoC: <https://www.llvm.org/OpenProjects.html>

[5] LLVM, AI Tool Use Policy: <https://llvm.org/docs/AIToolPolicy.html>

[6] Google, Writing a Proposal: <https://google.github.io/gsocguides/student/writing-a-proposal>